Reproducing LLM-as-a-Verifier — speaker script

李尚益 (Li Shangyi) · 1155270838 · FTEC5660

Timing: 12 slides, 15:00 of speech, 2054 words (≈137 words per minute — comfortable).

The same text is in each slide's **speaker notes**, so you can record from Presenters View. The 中文提示 line is a one-sentence reminder of what the slide is for; do not read it out.

Slide 1 — Title

`0:00 · 40s · 93 words`

Good afternoon. My name is Li Shangyi, student ID 1155270838, and this is my reproducibility work for FTEC5660 Agentic AI for Business and FinTech. I picked item twenty-one on the course list: LLM-as-a-Verifier, a general-purpose verification framework. My target is one row of its Terminal-Bench 2.1 table — best-of-three selection, where the paper reports that re-ranking five agent rollouts with a frozen verifier lifts accuracy from 79.4 to 86.5 percent. So the talk has three parts: what reproduced, the one thing I changed and what it did, and what broke along the way.

> **中文提示**：开场：自我介绍 + 选题（论文清单第 21 项）+ 目标行（TB 2.1 best-of-3）+ 三部分结构

Slide 2 — The claim under test

`0:40 · 70s · 184 words`

Let me state the claim precisely, because it is not a training result — it is a selection result. Terminal-Bench 2.1 gives, for each task, five rollouts from a small coding agent. The paper's claim is that a frozen verifier, which never sees whether a rollout succeeded, can pick the better one. On the paper's best-of-three configuration, the agent's own Pass@1 is 79.4 percent. After verifier selection it is 86.5, with a standard deviation of 1.1 over three seeds, against an oracle of 92.1 — the oracle being the fraction of tasks where at least one rollout happens to succeed. Best-of-five gives 78.7 to 88.0 against an oracle of 96.6. The mechanism has three parts, and I want you to hold on to all three: score each pair of rollouts on three criteria, use a twenty-letter scale, and — this is the part the paper emphasises — take the expectation of that score over the verifier's full logprob distribution instead of trusting one sampled label. Those pairwise rewards feed a pivot tournament, which is what keeps the number of comparisons linear instead of quadratic.

> **中文提示**：讲清这是“选择”而非训练；给出目标表；强调机制三要素（3 准则打分、20 字母刻度、**对 logprob 分布取期望**、pivot 锦标赛）

Slide 3 — Setup: what I actually ran

`1:50 · 80s · 166 words`

Now, what I actually ran — and where it differs from the paper, because that governs how you should read every number later. The verifier is DeepSeek's hosted model, deepseek-flash, which is V4.1 Flash, at temperature 1.0, with the top twenty token logprobs requested. The paper's self-

verification row uses a DeepSeek model as well, but a different build, so this is a controlled substitution: same tasks, same metric, same evaluation code, different judge. Two more deviations. First, scope: I use the first twelve of the twenty-four swing tasks, for reasons I will justify in a moment. Second, logprob availability — the primary backend does return token logprobs, but I will show you in a few slides that on this backend the distribution is degenerate, and a second backend that returns no logprobs at all reproduces the same scores through the text fallback. Everything else is the authors' own: their trajectories, their three criteria, their K equals two repeated verifications, one pivot, and their evaluation script, unchanged.

> **中文提示**：环境表：模型/解码/范围/配置；坦白三条偏差（同族不同 build、12/24 任务、logprob 可用性）；强调其余全是作者原装代码

Slide 4 — Step 0: the dataset and the metric check out exactly

`3:10 · 70s · 132 words`

Before spending a single API call, I checked the cheapest thing first: the ground-truth rewards that ship with the repository. Pass@1 and oracle need no verifier at all — they are arithmetic on those labels. Computing them on the full eighty-nine tasks gives 79.4 and 92.1 for best-of-three, and 78.7 and 96.6 for best-of-five. Those are the paper's numbers, to the decimal. Four out of four. That is the strongest positive result in this work, and it isolates the uncertainty: the trajectories, the pass-fail labels, the trial ordering and the metric definitions are all intact. So if anything diverges later, it diverges because of the verifier, and nothing else. It also confirms a convention that is otherwise invisible in the paper: best-of-three means the first three rollouts on disk, in file-name order.

> **中文提示**：不花 API 的第 0 步：用自带标签算 Pass@1/Oracle，四个数字全部逐位一致 → 数据与口径没问题，后面的偏差只能归因于 verifier；顺带揭示“best-of-3=磁盘前三个”的隐式约定

Slide 5 — Scope: a defensible subset, not cherry-picking

`4:20 · 80s · 198 words`

About the subset, because this is the first question I expect. Of the eighty-nine tasks, fifty-eight have all five rollouts succeeding and seven have all of them failing. On those, the verifier cannot change anything: the answer is decided before it is asked. Only the remaining twenty-four — the swing tasks, where the rollouts disagree — can move under selection. So the useful subset is already a third of the task list, and choosing it loses no coverage of the phenomenon; it only removes work that cannot affect the metric. Within those twenty-four I took the first twelve in sorted order. That is a mechanical rule — not "the easy ones" — and it was budget-driven: a full best-of-three over all twenty-four is about seven hundred verifier calls, and I wanted two configurations plus three seeds. Two honest caveats. Task-level accuracy now moves in steps of 8.3 points, and twelve is small. The subset is nevertheless representative: its Pass@1 is 50.0 percent against 52.9 for all twenty-four swing tasks, so it is not an easier or harder slice. Where twelve tasks are too coarse, I report comparison-level and trial-level statistics — thirty-one disagreeing comparisons and thirty-six scored trials.

> **中文提示**：回答“为什么要子集”：58 全过 + 7 全挂 的任务由标签决定，只剩 24 个 swing；取排序前 12（机械规则、非挑难度）；坦承 8.3 点步进与样本小，用 31 次比较 / 36 个 trial 补功效

Slide 6 — Result: does selection beat Pass@1?

`5:40 · 100s · 271 words`

Here is the answer to the headline question, and I want to be careful with how much it can carry. On the twelve tasks, the agent's Pass@1 is 50 percent. The verifier selects a successful rollout on eleven of twelve tasks — 91.7 percent — where the oracle is 100 percent, because every swing task has at least one success by definition. I ran three seeds, because the pipeline is stochastic: the verifier samples at temperature 1.0, and each seed also re-draws the tournament ring. All three seeds select eleven of twelve, so the mean is 91.7 with a standard deviation of zero. But I do not want you to stop at that number, because a twelve-task count moves in 8.3-point steps and hides almost everything. The finer metrics carry the signal: pairwise preference accuracy — how often the verifier prefers the successful rollout when the two rollouts disagree — is 90.3, 88.6 and 93.9 across the three seeds, so 90.9 plus or minus 2.7. Trial-level AUC is 0.841 plus or minus 0.022. Those three claims are statistically solid, not vibes. An exact sign test on eleven of twelve against the 50-percent baseline gives p equals 0.0032; on twenty-eight of thirty-one comparisons, p is 2.3 times ten to the minus six; and the AUC gives a z of 3.7, p equals 2 times ten to the minus four. So: the direction of the paper's claim reproduces, and it reproduces robustly. The magnitude — 86.5 plus or minus 1.1 — does not transfer, and I would not pretend it does: different verifier build, twelve tasks instead of sixty-six, and a hand-built subset.

> **中文提示**：结果：50% → 91.7%，三 seed 全 11/12（std=0）；强调任务级太粗，用逐对 90.9±2.7 与 AUC 0.841±0.022 承载信号，并给出三个检验 p 值

Slide 7 — The surprise: the expectation is inert

`7:20 · 120s · 196 words`

This is the most useful thing I found, and it changes what the modification can even test. The paper's reward is an expectation: the probability of each score letter times its value, summed. That only means something if the verifier actually spreads probability across letters. So I measured it. On the two hundred eighty-two comparisons that were scored under this configuration, the top token probability at the score position is 0.99999 — essentially one. Ninety-seven point two percent of the resulting scores sit within one thousandth of a scale point from a pure single letter, and the median deviation is seven times ten to the minus seven. The distribution is one-hot. Which means the expectation equals the argmax equals the sampled letter. The fine-grained machinery is running, but it is inert: on this backend, the method behaves as LLM-as-a-Judge with a twenty-point label. That is worth knowing on its own — anyone reusing this code on a hosted API should measure this before believing the fine-grained claim — and it reframes my modification. If the only surviving signal is the label alphabet, then the honest question is: does the resolution of that alphabet matter at all?

> **中文提示**：核心发现：实测 score token 上 top1≈0.99999、97.2% 的分数落在单字母 1e-3 刻度内、中位偏差 7e-7 → 分布 one-hot，期望=argmax=采样字母，细粒度机制失效（退化为 LLM-as-a-Judge）

Slide 8 — Modification: score granularity g20 → g5

9:20 · 80s · 182 words

That gives me an isolated, measurable modification: coarsen the score scale from twenty letters — A through T — to five, A through E. It is one change, in one file. The granularity is read from an environment variable and the scale description and the set of valid score tokens are built from it. Leave the variable unset and the scale is byte-identical to upstream — I checked that the description string and all forty valid tokens are unchanged, so the reproduction path above is untouched. Each configuration also writes its own score cache, so no score leaks from the baseline into the ablation. Why this and not the model swap? Because swapping the model would leave me with nothing to compare against — I would be measuring a different judge with no baseline. Coarsening the scale keeps the same judge, the same trajectories, the same criteria, and the same number of votes per task, and changes exactly one thing: how finely a preference can be written down. The result at task level: eleven of twelve in both configurations. No detectable difference.

> **中文提示**：修改：g20→g5，一个文件、一个环境变量，不设则与上游逐字节相同；两套配置各自 cache；解释为何不选“换模型”（没有同模型基线）；任务级 11/12 vs 11/12

Slide 9 — What changed, and what that means

10:40 · 100s · 221 words

But "no difference at task level" is not the same as "nothing happened", so I measured what the ablation did to the votes, on the same comparisons. Two hundred eighty-two comparisons were scored under both configurations. The mean absolute score change is 0.111 — a tenth of the scale — the preference margin widens from 0.317 to 0.369, and the number of distinct score values actually used collapses from twenty-two to eight. More tellingly, the pairwise preference flips. On comparisons where the two rollouts disagree — the ones that actually decide a task — it flips on 21 of 186, 11.3 percent. On comparisons where both rollouts share the same outcome — where a flip cannot matter — it flips on 29 of 96, 30.2 percent, almost three times as often. And yet the task decisions do not move, because each task's decision aggregates thirty votes: five directed comparisons, times three criteria, times two repetitions. The flips cancel. So the honest statement is not "granularity does not matter". It is "the tournament is robust to the label alphabet, but not blind to it" — and with twelve tasks, a difference of one task is invisible anyway: p equals 0.39. What I can rule out is a difference of two tasks or more, which would have shown up at p equals 0.019.

> **中文提示**：投票级影响：282 个同比较对，|Δ分数| 0.111、margin 0.317→0.369、档位 22→8；有信息比较翻转 11.3% vs 无信息 30.2%；因为每任务 30 票聚合所以抵消；功效上只能排除 ≥2 个任务的差异

Slide 10 — Debug diary: five real blockers

12:20 · 80s · 242 words

A short page on what actually cost the time, because in this kind of work it is most of the work. One: github dot com, raw dot githubusercontent and HuggingFace are unreachable from this machine's shell. The fix was to fetch from a CDN instead — but the repository is 440 megabytes and this reproduction needs 58, so I generated a manifest pinning 481 files by size and sha256, and a fetcher that verifies every file. The data is then reproducible without a working git remote. Two: the second API host returned Cloudflare error 1010 to every request. The cause was my probe's default User-Agent, not the API. Three — and this one matters — mid-run I hit HTTP 402, insufficient balance, and 429, concurrency above the limit. Failures are scored as 0.5-0.5 ties, which silently corrupts selection. The recovery is not "start over": failed calls are never written to the cache, so a re-run retries only the gaps, and you iterate until the error count is zero. That is why every number here comes from a zero-error run. Four: a backend with no token logprobs — the code path assumes them, and the generic OpenAI path depends on a vLLM-only trick that silently returns half-half. I added an env-gated fallback, justified by the measurement on the previous slide. And five, the boring one: 440 megabytes over a 45-kilobyte-per-second link. Two minutes for the 58 megabytes I needed, at the right parallelism.

> **中文提示**：调试日记：GitHub/HF 不通→CDN+sha256 清单；Cloudflare 1010→UA；402/429→失败不入缓存、重跑到 0 error；无 logprobs→env 开关+文本回退；440MB 慢链路→并行取子集

Slide 11 — Verdict: what reproduces

`13:40 · 50s · 81 words`

If I have to give a one-table verdict: the code runs, end to end, on two backends, with zero errors. The non-LLM numbers reproduce exactly. The direction of the verifier gain reproduces, and it is statistically solid. The published magnitude is not comparable — different model, fewer seeds, fewer tasks. And the fine-grained logprob expectation does not reproduce on this backend, because the distribution is one-hot; the granularity ablation then shows no detectable difference, which is exactly what that mechanism predicts.

> **中文提示**：判定表：代码能跑（两个后端 0 error）、非 LLM 数字完全复现、方向复现且有统计支撑、量级不可比、细粒度机制未复现、粒度消融无可检测差异

Slide 12 — Lessons + what I'd recommend

`14:30 · 30s · 88 words`

Three lessons I would pass on. Measure the score distribution before trusting a fine-grained reward — one cached batch is enough. Scope by swing tasks, because most of a benchmark cannot change under selection. And keep the score cache on disk, never caching a failure: that is what made a two-hour run survivable across two balance outages, and it is what lets anyone re-derive every number on this page without spending a token. Thank you. The repository is linked in the report; I am happy to take questions.

> **中文提示**：三条建议（先测分布、按 swing 任务剪裁、缓存不存失败）+ 致谢 + 邀请提问

Q&A 预案

Q: Why only twelve of the twenty-four swing tasks?

A: Because the assignment allows one clearly defined subset, and a full best-of-three over twenty-four tasks times two configurations times three seeds is about two thousand verifier calls. The subset is chosen by sorted name, not by difficulty, and its Pass@1 — 50.0 — sits within three points of the full swing-task value, 52.9. Extending it is one flag: the cache is per task, so no work is repeated.

Q: Why is the ablation single-seed while the baseline has three?

A: It is a matched-pair design. Both configurations share seed zero, so they score the same 282 of 288 comparisons — every comparison is its own control, and seed noise cancels in the paired difference. The baseline needs seeds because it is compared against the paper's mean and standard deviation.

Q: Isn't 91.7 versus 86.5 an improvement over the paper?

A: No — those numbers are on different task sets. Mine is twelve swing tasks where at least one rollout succeeds, so the ceiling is 100 percent; the paper's 86.5 is over sixty-six tasks including ones no rollout solves. Only the direction is comparable.

Q: The distribution being one-hot — is that your bug or the model's?

A: The model's, and it is measurable: top-token probability 0.99999 at the score position, median distance from a pure letter 7e-7. It is exactly why I stopped treating granularity as if the paper's mechanism were live, and why the two backends — one with logprobs, one without — are equivalent here.

Q: What would you do with more compute?

A: Two things, in order: the full twenty-four swing tasks so the verifier number is directly comparable to the paper's scope, and three seeds on the ablation for symmetry. A logprob-capable backend with real distribution mass is the prerequisite for testing the granularity claim properly.

Recorded demo you can add after slide 10 (no API cost):

cd ~/Desktop/repro/_repos/llm-as-a-verifier_llm-as-a-verifier

./.venv/bin/python repro/analyze.py --tags sub12_g20:20 sub12_g5:5

It replays the tournament from the checked-in caches and prints the table, in about two seconds.